

Căutare binară

Varianta clasică · Variații · STL · Complexitate
Căutare binară pe rezultat

Andrei Mihai Ciobanu

Algolymp Summer School

Cuprins

1. Căutarea binară a unui element
2. Variații
3. Variații în STL
4. Complexitate
5. Căutare binară pe rezultat
6. Recapitulare

Motivație

Problemă: într-un șir *sortat* cu n elemente trebuie aflat dacă valoarea x se află în el.

- **Căutare secvențială:** elementele se parcurg pe rând $\rightarrow$ în cel mai rău caz, n comparații

Întrebare

Cum poate fi folosit faptul că șirul este sortat pentru a face mai puține comparații?

- **Căutare binară:** se compară x cu elementul din mijloc, iar jumătatea în care x nu se poate afla se elimină la fiecare pas

n	Secvențial	Binar
10	10 pași	4 pași
100	100 pași	7 pași
1 000 000	10^6 pași	20 pași

Idee

Condiție

Șirul trebuie să fie **sortat** (crescător sau descrescător).

Se păstrează un interval $[st, dr]$ care conține răspunsul. La fiecare pas:

1. Se calculează mijlocul: $mid = (st + dr)\ /\ 2$ (împărțire întreagă)
2. Se compară $v[mid]$ cu x:

Întrebare

Ce se face în fiecare dintre cazurile $v[mid] = x$, $v[mid] < x$ și $v[mid] > x$?

- $v[mid] = x \rightarrow$ **găsit**
- $v[mid] < x \rightarrow st = mid + 1$ — se caută în dreapta
- $v[mid] > x \rightarrow dr = mid - 1$ — se caută în stânga

Oprire: Căutarea se oprește când valoarea este găsită sau când intervalul devine gol ($st > dr$).

Algoritmul pas cu pas

Șir sortat $v = [2,\ 5,\ 8,\ 11,\ 14,\ 17,\ 20,\ 23,\ 26,\ 29,\ 32]$, căutăm $x = 23$.

Întrebare

Câți pași sunt necesari pentru a găsi $x = 23$ și ce interval rămâne după fiecare pas?

2	5	8	11	14	**17**	20	23	26	29	32	$mid = 5$: $17 < 23 \to st = 6$
2	5	8	11	14	17	20	23	**26**	29	32	$mid = 8$: $26 > 23 \to dr = 7$
2	5	8	11	14	17	**20**	23	26	29	32	$mid = 6$: $20 < 23 \to st = 7$
2	5	8	11	14	17	20	**23**	26	29	32	$mid = 7$: $23 = 23$, găsit

Concluzie: Celula colorată este *mid*, iar celulele estompate sunt eliminate: intervalul $[st, dr]$ se înjumătățește la fiecare pas.

Căutare secvențială — implementare

```
int n, x, v[100005];     // se citesc n, x si v[0..n-1]

int poz = -1;
for (int i = 0; i < n; i++) {
    if (v[i] == x) {
        poz = i;
        break;
    }
}

cout << poz;
```

- Complexitate: $O(n)$ — în cel mai rău caz se parcurge tot șirul
- Funcționează pe orice șir, **sortat sau nu**

Căutare binară — implementare

```
int n, x, v[100005];     // se citesc n, x si v[0..n-1]

int st = 0, dr = n - 1, poz = -1;
while (st <= dr) {
    int mid = (st + dr) / 2;
    if (v[mid] == x) {
        poz = mid;
        break;
    } else if (v[mid] < x) {
        st = mid + 1;     // cautam in dreapta
    } else {
        dr = mid - 1;     // cautam in stanga
    }
}

cout << poz;
```

- Complexitate: $O(\log n)$

Întrebare

Ce se întâmplă dacă șirul nu este sortat?

Eliminarea unei jumătăți nu mai este justificată, deci algoritmul poate rata valoarea căutată.

Cuprins

1 Căutarea binară a unui element

2 Variații

3 Variații în STL

4 Complexitate

5 Căutare binară pe rezultat

6 Recapitulare

Lower bound și upper bound

Căutarea clasică găsește **o apariție oarecare** a lui x. Adesea este nevoie de:

- **Lower bound:** prima poziție cu $v[poz] \geq x$
- **Upper bound:** prima poziție cu $v[poz] > x$

Întrebare

Pentru $v = [1,\ 2,\ 2,\ 2,\ 3,\ 5]$ și $x = 2$, care sunt cele două poziții și câte apariții are x?

	Poziție	Valoare
Lower bound (≥ 2)	1	$v[1] = 2$
Upper bound (> 2)	4	$v[4] = 3$

Numărul de apariții: $\text{upper_bound} - \text{lower_bound} = 4 - 1 = 3$.

Implementare lower_bound

```
// prima pozitie cu v[poz] >= x
// ans = n daca toate elementele sunt < x
int st = 0, dr = n - 1, ans = n;
while (st <= dr) {
    int mid = (st + dr) / 2;
    if (v[mid] >= x) {
        ans = mid;      // candidat, continuam in stanga
        dr = mid - 1;
    } else {
        st = mid + 1;
    }
}
```

Când $v[mid] \geq x$, poziția se **salvează**, iar căutarea continuă mai la stânga, după un candidat mai bun.

Întrebare

Ce condiție trebuie modificată pentru `upper_bound`?

Singura diferență: condiția `v[mid] > x` în loc de `v[mid] >= x`.

Ultima poziție cu $v[poz] < x$ sau $\leq x$

Întrebare

Cum se obține ultima poziție cu $v[poz] \leq x$, folosind `upper_bound`?

- Ultima poziție cu $v[poz] < x = \texttt{lower_bound}(x) - 1$
- Ultima poziție cu $v[poz] \leq x = \texttt{upper_bound}(x) - 1$

Sau implementarea se adaptează direct, inversând sensul căutării:

```
// ultima pozitie cu v[poz] <= x
// ans = -1 daca toate elementele sunt > x
int st = 0, dr = n - 1, ans = -1;
while (st <= dr) {
    int mid = (st + dr) / 2;
    if (v[mid] <= x) {
        ans = mid;      // candidat, continuam in dreapta
        st = mid + 1;
    } else {
        dr = mid - 1;
    }
}
```

Pentru $< x$, condiția devine `v[mid] < x`.

Cuprins

1 Căutarea binară a unui element

2 Variații

3 Variații în STL

4 Complexitate

5 Căutare binară pe rezultat

6 Recapitulare

STL: `binary_search`, `lower_bound`, `upper_bound`

C++ oferă aceste funcții gata implementate în `<algorithm>`, toate cu complexitate $O(\log n)$.

Întrebare

Cum se află, cu aceste funcții, dacă x există în șir, prima lui poziție și numărul lui de apariții?

```cpp
#include <algorithm>
using namespace std;

int v[] = {1, 2, 2, 2, 3, 5};
int n = 6;

// Exista x in v? (true/false)
bool exista = binary_search(v, v + n, 2);

// Pozitii (indici)
int poz1 = lower_bound(v, v + n, 2) - v; // 1: prima aparitie
int poz2 = upper_bound(v, v + n, 2) - v; // 4: dupa ultima

// Numarul de aparitii ale lui x
int cate = poz2 - poz1;                  // 3
```

▷ *`lower_bound` întoarce un pointer / iterator; scăzând începutul tabloului se obține indicele*

STL — numărare pe interval $[x, y]$

Întrebare

Cum se numără elementele unui șir care se află în intervalul $[x, y]$?

Hint: Șirul se sortează, apoi se folosesc `lower_bound` pentru x și `upper_bound` pentru y.

```
int v[] = {23, 2, 5, 26, 11, 17, 14, 8, 20};
int n = 9;
sort(v, v + n);                 // sirul trebuie sortat inainte

int x = 8, y = 20;
int st = lower_bound(v, v + n, x) - v; // prima pozitie cu v[poz] >= x
int dr = upper_bound(v, v + n, y) - v; // prima pozitie cu v[poz] > y

int cate = dr - st;                    // 5: 8, 11, 14, 17, 20
```

- Idee frecventă: interogări pe un șir sortat în $O(\log n)$ în loc de $O(n)$

Cuprins

1 Căutarea binară a unui element

2 Variații

3 Variații în STL

4 Complexitate

5 Căutare binară pe rezultat

6 Recapitulare

Complexitatea căutării binare

La fiecare pas, intervalul $[st, dr]$ se **înjumătățește**:

$$n \to \frac{n}{2} \to \frac{n}{4} \to \cdots \to 1$$

Întrebare

După câți pași ajunge intervalul la un singur element?

Numărul de pași este $\log_2 n$.

n	$\log_2 n$	Pași maximi
10	3,32	4
1 000	9,97	10
10^6	19,93	20
10^9	29,90	30
10^{18}	59,79	60

Concluzie: Chiar și pentru $n = 10^{18}$ sunt necesari cel mult 60 de pași.

Cuprins

1 Căutarea binară a unui element

2 Variații

3 Variații în STL

4 Complexitate

5 Căutare binară pe rezultat

6 Recapitulare

Căutare binară pe rezultat — idee

- Căutarea binară clasică caută o **poziție** într-un tablou sortat
- Căutarea binară **pe rezultat** caută direct **valoarea răspunsului**, fără ca tabloul să existe

Întrebare

Ce informații sunt necesare pentru a căuta binar valoarea unui răspuns?

Schema în trei pași

1. Se stabilește intervalul $[st, dr]$ în care se află răspunsul
2. Se scrie o funcție $\mathrm{check}(x)$: „se poate rezolva problema cu valoarea x?"
3. Se verifică faptul că check este **monotonă**, apoi se caută pragul: cel mai mare x cu $\mathrm{check}(x)$ adevărat (sau cel mai mic, în varianta simetrică)

- Cuvinte-cheie în enunț: „cel mai mic / cel mai mare x astfel încât...", „minimizează maximul", „maximizează minimul"

▷ *Un număr din* $[1, 10^9]$ *se ghicește în cel mult* 30 *de întrebări „mai mare sau mai mic?", deoarece* $2^{30} > 10^9$

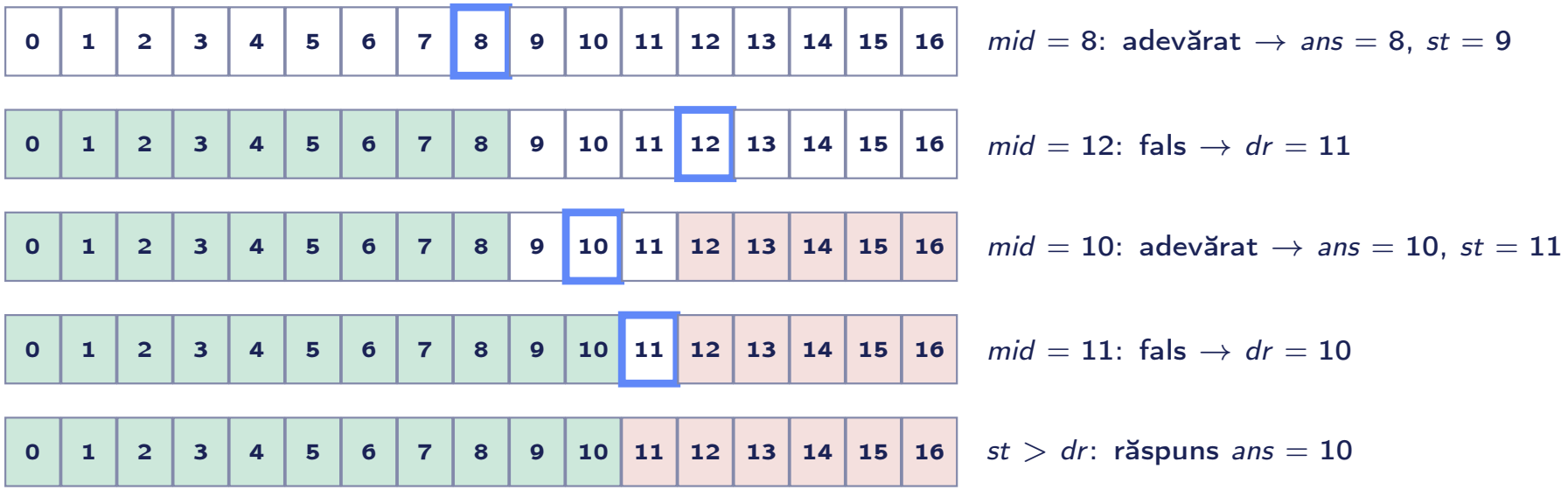
Căutare binară pe rezultat — desfășurare

Se caută cel mai mare x cu $\mathrm{check}(x)$ adevărat; aici $\mathrm{check}(x)$ este adevărat exact pentru $x \leq 10$. Intervalul necercetat $[st, dr]$ începe cu $[0, 16]$; celula cu margine albastră este *mid*.

Întrebare

Cum se modifică *st*, *dr* și *ans* după fiecare test?

Dacă $\mathrm{check}(mid)$ este adevărat, $ans = mid$ și $st = mid + 1$; altfel $dr = mid - 1$.



■ adevărat (cunoscut) ■ fals (cunoscut) □ necunoscut

Aici sunt 4 teste pentru 17 valori; pentru un domeniu de 10^{18} valori, circa 60 de teste.

Căutare binară pe rezultat — exemplu

Problemă

Se dau n bețe de lungimi $d_1, d_2, \ldots, d_n$. Din fiecare băț se taie bucăți de aceeași lungime întreagă L (restul rămas se aruncă). Să se determine **lungimea maximă** L pentru care se obțin în total **cel puțin** A bucăți.

Restricții: $1 \leq n \leq 10^5$, $1 \leq d_i \leq 10^9$, $1 \leq A \leq 10^9$.
Exemplu: $d = [7,\ 4,\ 9,\ 5]$, $A = 6 \Rightarrow L = 3$ (se obțin $2 + 1 + 3 + 1 = 7 \geq 6$ bucăți).

Întrebare

Cum se poate rezolva problema dacă se încearcă pe rând fiecare lungime posibilă, de la cea mai mare la cea mai mică?

Pentru fiecare L se calculează $\sum_i \lfloor d_i / L \rfloor$ și se oprește la prima valoare a lui L pentru care suma este $\geq A$. Complexitate $O(n \cdot maxim)$, unde $maxim$ este lungimea maximă a unui băț: pentru $maxim = 10^9$ și $n = 10^5$, mult prea lent.

Căutare binară pe rezultat — observație

Pe măsură ce L crește, numărul de bucăți obținute **scade** (sau rămâne la fel).
Exemplu: $d = [7,\ 4,\ 9,\ 5]$, $A = 6$

L	1	2	3	4	5	6	7	8	9
$\text{bucati}(L)$	25	11	7	5	3	2	2	1	1
$\geq A$?	✓	✓	✓	×	×	×	×	×	×

Întrebare

Ce proprietate are șirul valorilor $\text{check}(L)$, de la $L = 1$ spre dreapta?

Monotonie: Odată ce numărul de bucăți scade sub A, rămâne sub A. Condiția trece o singură dată de la adevărat la fals, deci la fiecare test se poate elimina jumătate din candidați.

Căutare binară pe rezultat — implementare

```
long long bucati(int d[], int n, int L) {
    long long cnt = 0;
    for (int i = 0; i < n; i++) {
        cnt += d[i] / L;
    }
    return cnt;
}

int st = 1, dr = maxim, ans = 0; // maxim = cel mai mare d[i]
while (st <= dr) {
    int mid = (st + dr) / 2;
    if (bucati(d, n, mid) >= A) {
        ans = mid;        // functioneaza, incercam mai mare
        st = mid + 1;
    } else {
        dr = mid - 1;     // nu functioneaza, incercam mai mic
    }
}
cout << ans;
```

- Complexitate: $O(n \log(maxim))$ — pentru aceleași date, $\approx 10^5 \cdot 30 = 3 \cdot 10^6$ operații

Overflow și condiția de aplicabilitate

Întrebare

De ce poate apărea overflow la $mid = (st + dr) \ / \ 2$ în căutarea binară pe rezultat?

Overflow

Intervalul $[st, dr]$ poate fi foarte mare, iar suma $st + dr$ poate depăși limita tipului `int` ($\approx 2{,}1 \cdot 10^9$). Se folosește `long long` sau `mid = st + (dr - st) / 2`. La căutarea unui element, $[st, dr]$ este mărginit de n, deci riscul nu apare de obicei.

Condiția de aplicabilitate

Trebuie să existe un **singur prag** în care check trece din adevărat în fals (sau invers). Verificare practică: dacă L este valid, este valid și $L - 1$?

- ✓ ✓ ✓ × × × — monotonă, se poate aplica
- ✓ × ✓ × ✓ — nu este monotonă, nu se poate aplica

Cuprins

1 Căutarea binară a unui element

2 Variații

3 Variații în STL

4 Complexitate

5 Căutare binară pe rezultat

6 Recapitulare

Recapitulare

Variantă	Timp	Spațiu
Căutare secvențială	$O(n)$	$O(1)$
Căutare binară (element)	$O(\log n)$	$O(1)$
STL `lower/upper_bound`	$O(\log n)$	$O(1)$
Căutare binară pe rezultat	$O(\log V \cdot \text{cost}(\text{check}))$	depinde

- Șirul trebuie sortat înainte de căutarea binară a unui element
- Pentru intervale $[st, dr]$ foarte mari se folosește `long long` sau `mid = st + (dr - st) / 2`
- `lower_bound` $\rightarrow$ prima poziție cu $\geq x$; `upper_bound` $\rightarrow$ prima poziție cu $> x$
- La căutarea binară pe rezultat, check trebuie să fie monotonă; complexitatea este $O(\log V \cdot \text{cost}(\text{check}))$, unde $V = dr - st$

Probleme de antrenament

Ușoare

1. **Classic Binary Search Practice** (AlgoCode #3963)
2. **First Position Not Less Than Query** (AlgoCode #4202)

Medii

1. **Maximum Bamboo Stake Length** (AlgoCode #1009)
2. **Snack Route** (AlgoCode #3962)

Grea

1. **Nth Number Coprime with 2022** (AlgoCode #1008)

Olimpiadă

1. **Buldo** (AlgoCode #1916 · OJI 2020 IX)
2. **Macarie** (AlgoCode #2984 · OJI 2024 IX)

Resurse

Română

1. **pbinfo.ro** — Căutarea binară (RO)
2. **infoas.ro** — Căutare binară în C++ (RO)

Engleză

1. **cp-algorithms.com** — Binary Search (EN, detaliat, include și căutarea binară pe numere reale)
2. **usaco.guide** — Binary Search on Monotonic Functions (EN)
3. **cp-algorithms.com** — Ternary Search (EN), căutare **ternară** pentru funcții unimodale

Documentație C++

1. **cppreference.com** — `lower_bound` / `upper_bound`